

L2 - 12.3. Network Monitoring with Zeek

Task 1 - Introduction

Every new beacon from `WKST-FINANCE-04` to the AS44477 infrastructure triggers an alert, indicating that the detection layer has finally caught up to where the attacker has been operating. What detection cannot do is fill in the hours that came before. The Cobalt Strike beacon from `10.14.22.88` had been running since `02:38 UTC` on `2025-11-14`, and the 5.3 MB HTTP POST to `185.213.154.201` happened at `03:15 UTC`. Snort started watching only after the rules were loaded, which means everything that happened before that point is sitting in a gap, waiting for us to read it.

The Finance subnet perimeter includes a Zeek sensor. Zeek is a passive network monitor that watches, parses, and writes structured log entries for every connection, every DNS query, every TLS handshake, and every file transfer it sees. Each protocol analyser produces its own log file, and every entry from the same network session shares a unique identifier called a `uid`. An analyst who follows that `uid` across the log set can reconstruct the entire session without ever touching a packet, and that is the investigative pattern this room will teach.

- `logs/`: the six pre-generated Zeek logs
(`conn.log`, `dns.log`, `http.log`, `ssl.log`, `files.log`, `notice.log`)
- `pcaps/finance-sensor.pcap`: the upstream packet capture behind those logs
- `scripts/dns_notice.zeek`: the starter detection script we will edit later in the room
- `threat-feed.csv`: the reference threat feed used in the file-hash exercise

Every command in this room runs from `/home/ubuntu`, so we will reference these artifacts using relative paths like `logs/conn.log`, `pcaps/finance-sensor.pcap`, and `threat-feed.csv`.

Task 2 - Reading Connection Records with Zeek

Every Zeek based investigation starts in conn.log. If Zeek observed a TCP, UDP, or ICMP session, there is a row for it in this file. Each row carries the source and destination, the bytes that moved in each direction, the duration, and a state code that tells us how the session ended.

conn_state Quick Reference

The single most useful column after the IP addresses

S0 SYN sent, no response <i>Many to many destinations = scan signature.</i>	SF Established and cleanly closed <i>Repeated clean sessions = beaconing clue.</i>	REJ RST in response to SYN
OTH No SYN observed, mid-session <i>Use byte counts cautiously.</i>	S1 Established, close not seen	RSTO Originator RST after handshake

The Zeek logs are tab-separated text files with a header section that names each column, so let us begin by looking at the header itself, as shown below: Inspect conn.log Header

```
ubuntu@tryhackme:~$ head -8 logs/conn.log
#separator \x09
#set_separator ,
#empty_field (empty)
#unset_field -
#path conn
#open 2026-05-04-17-31-44
#fields ts uid id.orig_h id.orig_p id.resp_h id.resp_p
proto service duration orig_bytes resp_bytes conn_stat
e local_orig local_resp missed_bytes history orig_pkts
orig_ip_bytes resp_pkts resp_ip_bytes tunnel_parents i
p_proto
#types time string addr port addr port enum
string interval count count string bool bool c
```

```

count    string    count    count    count    count    set[string] c
count    string    string    string    string

```

The `#fields` line lists every column name in order, and the columns we will work with most are `ts`, `uid`, the `id.*` quartet (source IP and port, destination IP and port), `proto`, `service`, `duration`, `orig_bytes`, `resp_bytes`, and `conn_state`.

The `#unset_field` value `-` is what Zeek writes when a field is absent, which becomes important when we read the User-Agent column in `http.log` later in the room. We will be using `zeek-cut` to pick the columns we want, as shown below: First Look at conn.log

```

ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.resp_h id.re
sp_p proto service duration orig_bytes resp_bytes conn_state
< logs/conn.log | head -5
1763087400.000000 CE0nuI3nDlaV4XL4x8 10.14.22.88 10.0.0.53
53 udp dns 0.005000 28 54 SF
1763087430.598559 CDq4fC1kEgsWrHHWbd 10.14.22.88 10.0.0.53
53 udp dns 0.005000 32 62 SF
1763087449.432986 CW04ZvZ9dq02SQyw4 10.14.22.88 10.0.0.53
53 udp dns 0.005000 36 70 SF
1763087472.619239 C9A1eg2W7AGrD0J7Wc 10.14.22.88 10.0.0.53
53 udp dns 0.005000 32 62 SF
1763087515.047345 CtayFt2fPHBjYhIF59 10.14.22.88 10.0.0.53
53 udp dns 0.005001 28 54 SF

```

Looking at the output, the first rows are baseline DNS resolution from `WKST-FINANCE-04` to the internal DNS server on UDP/53, all closing cleanly with the `SF` state and showing small byte counts on both sides. This is normal workstation behaviour, and it is the surface against which we will measure suspicious activity in the rest of the file.

The conn_state Field: Connection state is one of the first fields we use during analysis. The common states below explain how Zeek saw each session end:

State	Meaning	Significance
S0	SYN sent, no response	Port scan, firewall drop, closed port
SF	Established and cleanly closed	Expected for completed sessions
REJ	Rejected (RST in response to SYN)	Firewall block, ACL deny
OTH	No SYN observed, mid-session data	Asymmetric capture
S1	Established, close not observed	Possibly active, check duration
RST0	Originator RST after established	Client-side abort

It is important to note that a flood of S0 from a single source to many destinations on the same port is a port-scan signature, while many sources sending S0 to many destinations indicates distributed scanning or botnet behaviour. A clean-close state with regular timing and consistent byte counts is the signature of beaconing. OTH on a host under investigation tells us that our visibility is incomplete and the byte counts cannot be trusted, so we will flag it in the analysis and look for what may have been missed.

Hunting Beacons by Session Count: Beaconing produces the same source-destination-port triplet over and over, so the cleanest way to surface that pattern is to count how often each triplet appears. Let us use the following command to rank those repeated pairs: Rank Connection Pairs by Count

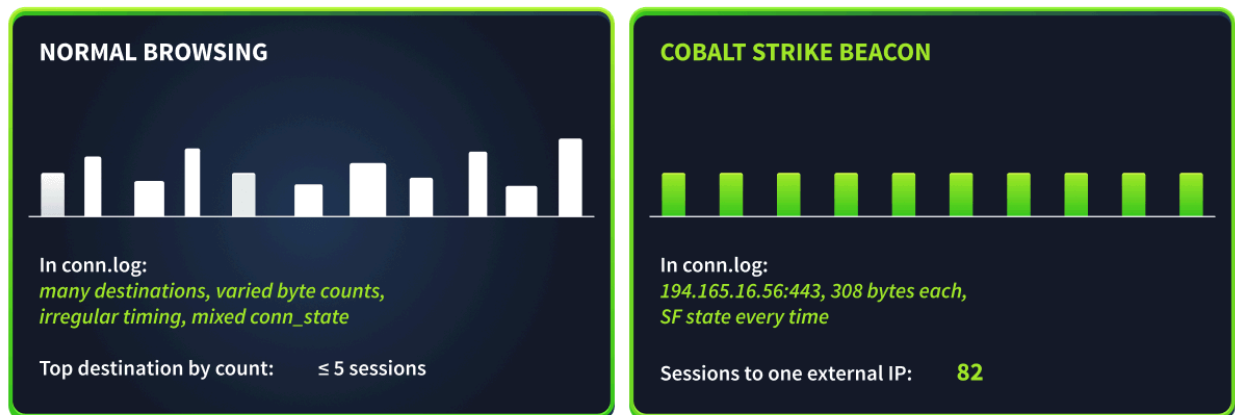
```
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h id.resp_p <
logs/conn.log | sort | uniq -c | sort -rn | head -5
196 10.14.22.88 10.0.0.53 53
82 10.14.22.88 194.165.16.56 443
15 10.0.0.53 1.1.1.1 53
1 10.14.22.88 185.220.101.45 80
1 10.14.22.88 185.213.154.201 80
```

Looking at the output, the first row is normal DNS resolution from WKST-FINANCE-04 to the internal resolver, but the row that does not belong is the second one: 82 sessions from 10.14.22.88 to 194.165.16.56 on TCP/443. No legitimate workstation makes 82 connections to a single external IP on TCP/443 within a 90-minute

window, because a normal browser distributes its traffic across many destinations. So 82 repeated connections to one external IP is the signature we are looking for.

conn.log Reveals the Beacon

82 short sessions to one IP, 60 seconds apart



No legitimate workstation makes 82 connections to one external IP in 90 minutes.

To confirm the interval, let us sort the matching sessions by timestamp and read the gaps between them: Beacon Timing Pattern

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h id.resp_h id.resp_p
duration orig_bytes < logs/conn.log | awk -F'\t' '$3=="194.16
5.16.56" && $4=="443"' | sort -k1 | head -5
1763087880.000000 10.14.22.88 194.165.16.56 443 0.044000
308
1763087942.388062 10.14.22.88 194.165.16.56 443 0.044000
308
1763088002.918872 10.14.22.88 194.165.16.56 443 0.043999
308
1763088062.344944 10.14.22.88 194.165.16.56 443 0.043999
308
1763088117.959790 10.14.22.88 194.165.16.56 443 0.043999
308
```

Looking at the timestamps, the gap between consecutive sessions sits at roughly 60 seconds with a small jitter, and the duration of every session is 0.044 seconds

with `orig_bytes` at 308 every time. The same shape repeats hour after hour, which is the signature of a Cobalt Strike check-in beacon rather than typical application traffic.

Now let us confirm the connection state these beacons close in by filtering the same source-destination pair and listing the unique state values: Confirm Beacon Connection State

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h id.resp_p co
nn_state < logs/conn.log | awk -F'\t' '$2=="194.165.16.56" &&
$3=="443"' | sort -u
10.14.22.88 194.165.16.56 443 SF
```

A single unique row comes back with `conn_state` of `SF` for every beacon, which means every one of those sessions establishes cleanly and closes cleanly. The malware is not hiding the connection at the TCP layer; it is hiding the activity inside dozens of look-alike short HTTPS sessions that look indistinguishable from a normal web request at the connection level.

Spotting Internal Scanning: Soon after a beacon establishes, the operator typically looks for adjacent hosts to pivot to. The pattern in `conn.log` is one source touching many internal IPs on the same port and getting no response on most of them, which produces a wall of `S0` rows that we can filter for directly, as shown below: Internal Scan Count

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h id.resp_p co
nn_state < logs/conn.log | awk -F'\t' '$1=="10.14.22.88" &&
$3=="445" && $4=="S0"' | wc -l
23
```

The count comes back at 23, which means 23 TCP connections from `WKST-FINANCE-04` to internal hosts on port 445, every one of them in `S0` state. The port is SMB, the state means no response was observed, and the count tells us the actor is fingerprinting the segment looking for shares to pivot through. None of these scans appear in `dns.log`, `http.log`, or `ssl.log`, because the activity is purely TCP probes and `conn.log` is the only log on the box that sees them.

Finding the Largest Outbound Transfer: Beacons are small, but when real data leaves the network the byte volume changes shape immediately, so let us sort `conn.log` by `orig_bytes` and look for outliers at the top of the list: Largest Outbound Transfer

```
ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.resp_h id.re
sp_p orig_bytes resp_bytes < logs/conn.log | awk -F'\t' '$3=
="10.14.22.88" && $4!~/^10\.14\. / {print $0}' | sort -t'\t'
-k6 -rn | head -3
1763090107.000000 CmQW1d3iuUHHDgqYe7 10.14.22.88 185.213.1
54.201 80 5348913 110
1763092746.812696 CnWRcG1BoLkvUXHYoh 10.14.22.88 194.165.1
6.56 443 308 419
1763092690.911542 Cqgcyq3AM37D2nmh0l 10.14.22.88 194.165.1
6.56 443 308 419
```

Looking at the output, one connection dominates the rest by a good margin. The source is `10.14.22.88`, the destination is `185.213.154.201` on port 80, and `orig_bytes` is `5348913`, which tells us roughly 5 MB left the network in a single HTTP session. The remaining rows in the top three are beacons at 308 bytes each, and they vanish in comparison to the exfiltration row above them.

The `orig_bytes` field counts everything the originator sent at the application payload layer, which includes the HTTP request headers as well as the body. The actual file body inside the POST is 5,348,721 bytes and the additional 192 bytes are the HTTP headers wrapping the upload, and we will see the body length on its own when we move into `http.log` in the next step.

Two findings stand out from this single log file: `WKST-FINANCE-04` is beaconing every minute to `194.165.16.56` on TCP/443, and the same host pushed roughly 5 MB to `185.213.154.201` on TCP/80 in one connection. We have not opened a packet, loaded an HTTP analyser, or touched a TLS handshake, but we have a query, an interpretation, and a clear escalation, which is the value `conn.log` provides as the first stop in any Zeek-based investigation.

`zeek-cut` is a specialized command-line utility bundled with the Zeek Network Security Monitoring platform. It extracts and formats specific columns from TSV (tab-separated value) log files generated by Zeek.

`conn.log` is the core log file generated by the **Zeek** Network Security Monitoring platform. It tracks every IP-level network transaction observed on the wire, summarizing each connection into a single log entry containing network identifiers, timing metrics, bytes transferred, and state flags.

Answer the questions below

What is the **first internal host** that `10.14.22.88` attempts to reach on TCP/445 with `conn_state` `S0` ?

10.14.22.10

```
zeek-cut ts id.orig_h id.resp_h id.resp_p conn_state < logs/conn.log \
| awk -F'\t' '$2=="10.14.22.88" && $4=="445" && $5=="S0"' | sort -k1 -n | head -3
```

`conn.log` records every TCP, UDP, and ICMP connection detected by Zeek, while `zeek-cut` extracts targeted columns. Since beaconing involves recurring communication between the same source, destination, and port, tracking these identical triplets quickly exposes the behavior.

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h id.resp_h id.resp_p conn_state < logs/conn.log \
| awk -F'\t' '$2=="10.14.22.88" && $4=="445" && $5=="S0"' | sort -k1 -n | head -3
1763090100.000000      10.14.22.88      10.14.22.10      445      S0
1763090100.078552      10.14.22.88      10.14.22.11      445      S0
1763090100.283725      10.14.22.88      10.14.22.12      445      S0
```

What is the last timestamp observed for the repeated TCP/443 sessions from 10.14.22.88 to 194.165.16.56?

1763092746.812696

```
zeek-cut ts uid id.orig_h id.resp_h id.resp_p orig_bytes resp_bytes < logs/conn.log
| awk -F'\t' '$3=="10.14.22.88" && $4!~/^10\.14\. / {print
```

```
$0}' | sort -t$'\t' -k6 -rn | head -3
```

The two requested items target exact data points rather than contextual context. Filtering for the scan signature and sorting chronologically, rather than relying on arbitrary default output, pinpoints 10.14.22.10 as the first local target the endpoint attempts to reach on TCP/445 with `conn_state=50`:

```
ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.resp_h id.resp_p orig_bytes res
p_bytes < logs/conn.log | awk -F'\t' ' $3=="10.14.22.88" && $4!~/^10\.14\. / {prin
t $0}' | sort -t$'\t' -k6 -rn | head -3
1763090107.000000      CmQW1d3iuUHHDgqYe7      10.14.22.88      185.213.154.2018
0      5348913 110
1763092746.812696      ChyYn01kLVeL7Ux5ik      10.14.22.88      194.165.16.56 4
43      308 419
1763092690.911542      CVZ29lQ1Xk6opzli6      10.14.22.88      194.165.16.56 4
43      308 419
```

Task 3 - DNS Visibility and Tunnelling Detection

DNS Tells Us Where a Connection Was Going: Every external connection from a workstation begins with a DNS query, because before TCP opens and before TLS is negotiated, the host must resolve a name to an address. `dns.log` is the record of those resolutions, and for an analyst it is the answer to one of the most useful investigative questions: what domain was the host trying to reach, and what did that domain resolve to?

In the previous task, we learned that `conn.log` showed us the beacon to `194.165.16.56` and the exfiltration POST to `185.213.154.201`. Still, neither of those connections tells us the domain the malware was using to find them. `dns.log` does, and that is what makes the cross-correlation between the two logs valuable.

Three DNS Queries, Three Stories

Length alone is not the signal - source, SLD, and qtype are

NORMAL QUERY

10.14.22.88 → www.microsoft.com

DNS TUNNEL CANDIDATE

10.14.22.88 → z6s5vtgvxm7vpdfdirfbjuef56w4b7uv2lqguwxhj3vpejgux.exfil-channel.net

DNSSEC VALIDATION (LEGITIMATE)

10.0.0.53 → ad31fb585692bc49f060f7bd7e8557f4abc17a0c52ae797f4b0557d1.dnssec-validation.local

Reading dns.log: The schema is similar to `conn.log`: tab-separated, header at the top, and we will be using `zeek-cut` again to extract the columns we want. The fields we will care about most are `ts`, `id.orig_h` (the querying host), `query` (the domain), `qtype_name` (the record type), `rcode_name` (the response code), and `answers` (the resolved IPs). First Look at dns.log

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h query qtype_name rc
ode_name answers < logs/dns.log | head -10
1763087400.000000 10.14.22.88 github.com A NOERROR 151.1
01.190.120
1763087430.598559 10.14.22.88 wiki.nfg.local A NOERROR 1
0.14.0.75
1763087449.432986 10.14.22.88 intranet.nfg.local A NOERR
OR 10.14.0.54
1763087472.619239 10.14.22.88 api.github.com A NOERROR 1
51.101.165.151
1763087515.047345 10.14.22.88 github.com A NOERROR 151.1
01.216.30
1763087520.000000 10.0.0.53 ad31fb585692bc49f060f7bd7e855
7f4abc17a0c52ae797f4b0557d1.dnssec-validation.local DS NOERR
OR DS 48 46
1763087541.277349 10.14.22.88 www.microsoft.com A NOERR
```

```

OR 151.101.218.74
1763087566.621425 10.14.22.88 github.com A NOERROR 151.1
01.85.159
1763087591.313790 10.14.22.88 wiki.nfg.local A NOERROR 1
0.14.0.13
1763087615.737583 10.14.22.88 api.github.com A NOERROR 1
51.101.35.80

```

Looking at the output, the first few rows are mostly normal: the workstation resolving `github.com`, `microsoft.com`, and the internal `nfg.local` zone. One row stands out, though, which is the entry from `10.0.0.53`: the query is a long hex-looking label under `dnssec-validation.local`, the record type is `DS`, and the source is the internal DNS server itself. We will come back to this row when we get to the scripting task, and for now we will just note that long DNS labels exist on this network for legitimate reasons.

Resolving the C2 Domain: We have a suspect IP from `conn.log`: `194.165.16.56`. The investigative move is to ask `dns.log` what name the workstation looked up to reach it, and we will do that by searching backwards from the resolved IP to the queried domain, as shown below: Resolve the C2 Domain

```

ubuntu@tryhackme:~$ zeek-cut id.orig_h query answers < logs/d
ns.log | grep "194.165.16.56"
10.14.22.88 update.<REDACTED>.com 194.165.16.56

```

Looking at the output, we get a single match, which tells us `WKST-FINANCE-04` queried `update.<REDACTED>.com` and the resolver returned `194.165.16.56`. That domain is the attacker's C2 hostname, and the name is picked to look like a software-update CDN even though no such CDN exists, with the IP it points at sitting in AS44477 (Stark Industries Solutions), a hosting provider with a long history of malicious infrastructure. Two findings come together in one row: the C2 IP, and the domain the malware uses to reach it.

Long Labels on the Wire: DNS tunnelling encodes data inside the subdomain portion of a query. Because DNS labels can be up to 63 characters each and a full domain can be up to 255 characters, an attacker can move kilobytes of encoded data per minute through nothing more than DNS queries, and the signature

in `dns.log` is queries with abnormally long labels repeated to the same second-level domain. Let us hunt for queries longer than 60 characters total, as shown below: Long-Label Query Hunt

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h query < logs/dns.log |  
awk -F'\t' 'length($2) > 60' | sort | uniq -c | sort -rn | he  
ad -10  
1 10.14.22.88 z6s5vtgvxm7vpdfdirfbjuef56w4b7uv2lqguwxhj3vpe  
jguvx.exfil-channel.net  
1 10.14.22.88 vbxvvsds46dsuw7qq6nypnx5n2xh7fxsaxi7uehq2  
7clq.exfil-channel.net  
1 10.14.22.88 sigsbcwxuoprimngnuyhlp3dvqgds2o5ow662rgn  
4zh3lg3l4bbye.exfil-channel.net  
1 10.14.22.88 k7qpdombl3lwoluibpiizfnhu47z5nj32t23q2mpj  
dq6c6a42srt.exfil-channel.net  
1 10.14.22.88 jrbhwqhuvbgtnpdgrkaoqv522dmkxf4htbrqqjm2u  
4dli3apy.exfil-channel.net  
1 10.14.22.88 ilnuzahalrwm3ztng55vttt6jqu76hneu3re4e7f  
e2qjjmew.exfil-channel.net  
1 10.14.22.88 hpxlij7sbfxvgwgpy43rlkivp4xisiuts5vmyqrue  
lgmji7jnx4ci7r.exfil-channel.net  
1 10.14.22.88 fxjaozl4d6qrfwcf2xrmaucsq3x343y46hxvo7bas  
zy3tq.exfil-channel.net  
1 10.14.22.88 fb3juskh46huo7dyyeujifurlbjvtpemthwjwk7qi  
i5njjynn2y6k.exfil-channel.net  
1 10.14.22.88 duweseivvd27d4wmwsuf57rpvuqw674g5ei3vdpc  
a5k6r5s.exfil-channel.net
```

Looking at the output, every row from `WKST-FINANCE-04` follows the same structure: a long Base32-style label of letters and digits followed by a constant second-level domain `exfil-channel.net`. The constant SLD is the attacker's infrastructure and the varying label is encoded data, which makes this iodine-style or dnscat2-style DNS tunnelling that is happening in parallel with the HTTPS beaconing we identified in the previous task. To confirm `10.14.22.88` is the only host responsible for these long queries (excluding the legitimate DNSSEC traffic from the internal resolver), let us count per source: Tunnel Source Count

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h query < logs/dns.log |
awk -F'\t' '$1!="10.0.0.53" && length($2) > 60 {count[$1]++}
END {for(h in count) print count[h], h}' | sort -rn
18 10.14.22.88
```

`10.14.22.88` produces all 18 of the long-label tunnel queries on this dataset, and no other workstation is doing it, which makes the pattern a per-host indicator we can pivot on across the rest of the investigation.

The Domain Behind the Exfiltration: Let us apply the same backward correlation for the exfil destination from `conn.log`, where the IP this time is `185.213.154.201`: Exfil Domain Resolution

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h query answers < log
s/dns.log | grep "185.213.154.201"
1763090096.000000 10.14.22.88 backup.<REDACTED>.com 185.2
13.154.201
```

A single row comes back, timestamped at `1763090096` (which is 03:14:56 UTC on 2025-11-14). The host queried `backup.<REDACTED>.com` and the resolver returned `185.213.154.201`, and eleven seconds later the HTTP POST to that IP began. The domain is picked to look like an internal backup-sync service and the timing places the resolution immediately before the data transfer, so both signals together strengthen the verdict that the 5.3 MB POST was an exfiltration rather than legitimate activity.

Distinguishing Genuine Tunnels From Legitimate Long Queries: Before we leave `dns.log`, let us look at the row from `10.0.0.53` we noted in the first sample. The internal DNS server runs DNSSEC validation, which produces queries for hashed NSEC3 record names. These queries are long, hex-looking, and returned `NOERROR`: DNSSEC Validation Sample

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h query qtype_name < log
s/dns.log | awk -F'\t' '$1=="10.0.0.53" && length($2) > 60' |
head -3
10.0.0.53 ad31fb585692bc49f060f7bd7e8557f4abc17a0c52ae797f4
```

```
b0557d1.dnssec-validation.local DS
10.0.0.53 89f8d4299fbd329be2d37522aa1dd2a1152eec292be7c837f
a8f2f39.dnssec-validation.local DS
10.0.0.53 2fee0a735124788c213c42a6633abef777066836f39486979
e31aa60.dnssec-validation.local DS
```

Looking at the output, the shape is similar to the tunnelling traffic but the source, the SLD, and the query type are all different. Source `10.0.0.53` is the internal DNS server, the SLD is `dnssec-validation.local`, and the `qtype_name` is `DS`. None of those match the workstation tunnel pattern of source `10.14.22.88`, SLD `exfil-channel.net`, and qtype `TXT`, which is what makes the two patterns separable in the first place.

This distinction matters because a naive long-label rule (one that fires on any query over 60 characters) catches both the genuine tunnel and the legitimate DNSSEC traffic, so a useful rule has to suppress the legitimate source while keeping the genuine signal. We will write that rule and apply that suppression in the scripting task later in the room.

Answer the questions below

What DNS record type is used by the long `exfil-channel.net` queries from `10.14.22.88`?

txt

The DNS record type used by the long **exfil-channel.net** queries from **10.14.22.88** is **TXT**.

A DNS query immediately precedes the 03:15 UTC HTTP POST exfiltration. What is the full domain name of that query?

backup.corpfiles-sync.com

The query at **03:14:56 UTC** (`1763090096.000000`) for `backup.corpfiles-sync.com` resolves the destination IP (`185.213.154.201`) where the HTTP POST exfiltration is immediately sent at **03:15 UTC**.

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h query answers < logs/dns.log | grep "1
85.213.154.201"
1763090096.000000      10.14.22.88      backup.corpfiles-sync.com      185.213.
154.201
```

Which domain in `dns.log` resolved to the IP address 194.165.16.56?

update.softpatch-cdn.com

Resolves to Known-Bad Infrastructure: In the DNS telemetry logs (`logs/dns.log`), the lookup for `update.softpatch-cdn.com` from infected workstation `10.14.22.88` resolved directly to the malicious C2 IP address `194.165.16.56`.

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h query answers < logs/dns.log | grep "194.165.16.56"
10.14.22.88      update.softpatch-cdn.com      194.165.16.56
```

Task 4 - HTTP and TLS Protocol Logs

What HTTP and TLS Reveal: `conn.log` told us a connection happened, but it did not tell us what was inside. The protocol-specific logs do. `http.log` records every HTTP transaction Zeek observed, with the method, URI, headers, byte counts, and response code. In contrast, `ssl.log` records TLS handshakes, including the negotiated version and the SNI value in the `server_name` field.

Two findings from the previous step now need protocol-level context. The 5.3 MB POST to `185.213.154.201` should appear in `http.log` because the connection was on TCP/80, and the repeated connections to `194.165.16.56:443` should appear in `ssl.log` because TCP/443 to an external IP is usually TLS.

The Exfil POST in http.log: `http.log` for this dataset is small, with two rows in total, as shown below: First Look at http.log

```
ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.resp_h method host uri user_agent request_body_len status_code < logs/http.log
1763088600.061000 CdxY4e40hSKGjp4mDf 10.14.22.88 185.220.101.45 GET fileshare.corp-helpdesk.net /downloads/invoice_march.pdf \x2d 0 200
1763090107.081000 CmQW1d3iuUHHDgqYe7 10.14.22.88 185.213.154.201 POST 185.213.154.201 <REDACTED> - 5348721 200
```

Two transactions are visible: a `GET` for what appears to be a PDF download, and a `POST` to `185.213.154.201` from `WKST-FINANCE-04`. The POST is the row we care about now. Reading the POST row left to right, we see timestamp `1763090107` (03:15:07

UTC), uid `CmQWld3iuUHHDgqYe7`, method `POST`, host `185.213.154.201`, URI `<REDACTED>`, user_agent `-`, request body length `5348721`, and status code `200`.

Three findings come together here. The method is `POST` with a 5.3 MB body, which means data is being uploaded rather than downloaded. The Host header is a bare IP address, which means the client did not even pretend to be talking to a named service. The User-Agent is missing, which is unusual because browsers and most legitimate clients send one. Together, they describe a custom client that uploads a multi-megabyte payload to a bare IP address. Let us isolate every empty User-Agent on the wire with a simple filter: Empty User-Agent Hunt

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h method uri user_agent request_body_len < logs/http.log | awk -F'\t' '$5==\"-\"'
```

10.14.22.88	185.213.154.201	POST	<REDACTED>	-	5348721
-------------	-----------------	------	------------	---	---------

The exfil POST is the only row with a clean User-Agent value in this dataset. In a production environment, this filter is a useful triage query against `http.log`, because legitimate browsers and frameworks almost always identify themselves.

ssl.log and the C2 Beacon: The beacons we identified in the previous step were on TCP port 443 and originated from `194.165.16.56`. Each one of those connections should produce a TLS handshake, and `ssl.log` should have a row for each observed handshake. Let us read the first five, as shown below: First Look at `ssl.log`

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h id.resp_h version server_name < logs/ssl.log | head -5
```

1763087880.041000	10.14.22.88	194.165.16.56	TLSv12	-
1763087942.429062	10.14.22.88	194.165.16.56	TLSv12	-
1763088002.959872	10.14.22.88	194.165.16.56	TLSv12	-
1763088062.385944	10.14.22.88	194.165.16.56	TLSv12	-
1763088118.000790	10.14.22.88	194.165.16.56	TLSv12	-

Looking at the output, every visible beacon shows `TLSv12` and a `server_name` value of `-`. The `server_name` field records SNI from the TLS Client Hello. In normal browser traffic, the client usually sends the hostname it is trying to reach so the

server can choose the right certificate. When `server_name` is `-`, Zeek did not observe an SNI value.

For external HTTPS traffic, repeated connections to a bare IP with no SNI are worth investigating. A single empty SNI can occur with older clients, custom tooling, or unusual configurations, but the pattern here is the same: the same workstation repeatedly connects to `194.165.16.56` over TLS and never presents a hostname. That lines up with the beaconing pattern we already saw in `conn.log`. Let us confirm that an empty SNI is consistent across the C2 destination: Confirm Empty SNI

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h server_name
< logs/ssl.log | awk -F'\t' '$2=="194.165.16.56" | sort -u
10.14.22.88 194.165.16.56 -
```

A single unique row comes back. Every TLS session to `194.165.16.56` shows the same source and destination addresses and an empty SNI. That gives us protocol-level support for the beacon pattern from `conn.log`. Now let us count the number of TLS sessions to `194.165.16.56` where `server_name` is empty: Count Empty SNI Sessions

```
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h server_name
< logs/ssl.log | awk -F'\t' '$2=="194.165.16.56" && $3=="- "
{c++} END{print c+0}'
<REDACTED>
```

The count is `<REDACTED>`, matching the repeated connection pattern from `conn.log`. We now have two independent signals for the same activity: repeated clean TCP sessions to a single external IP and repeated TLS handshakes to that IP without SNI. Together, they support the conclusion that this is automated beacon traffic rather than normal browsing.

MIME From the Magic Bytes: The exfil POST in `http.log` shows a `request_body_len` of 5,348,721 bytes, but the column does not tell us what those bytes are. For that, we use `files.log`, which records files that Zeek extracted from network traffic and identifies their type from the magic bytes at the start of each file. We pivot via the uid: MIME From `files.log` via uid

```
ubuntu@tryhackme:~$ grep -E "(^#)|(CmQWld3iuUHHDgqYe7)" logs/
files.log | zeek-cut source mime_type filename total_bytes
HTTP      application/<REDACTED>  backup_archive.zip  5348721
HTTP      -      -      7
```

A **MIME mismatch** in Zeek occurs when the file type declared by a application protocol (like an HTTP `Content-Type` header or email attachment metadata) conflicts with the actual file type detected by Zeek's file analysis framework.

Two file rows appear for the same uid because the connection carried both the upload and the small response. The first row is the request body: HTTP protocol, MIME `application/<REDACTED>`, filename `backup_archive.zip`, and size `5348721` bytes. The mime type comes from Zeek reading the first bytes of the file (`50 4B 03 04`, the ZIP magic), regardless of what the HTTP `Content-Type` header claimed.

Three layers of the same event now line up. `conn.log` reported 5.3 MB went outbound, `http.log` reported it was a POST to `<REDACTED>` with no User-Agent, and `files.log` reported it was a ZIP archive named `backup_archive.zip`. We have enough to describe the event without opening the packet capture.

Answer the questions below

What is the value of the `uri` field in `http.log` for the POST from 10.14.22.88 to 185.213.154.201?

/upload/data

Because data was exfiltrated via an unencrypted HTTP POST request, `http.log` provides complete application-layer visibility.

```
zeek-cut uid id.orig_h id.resp_h method uri request_body_len
< logs/http.log \
| awk -F'\t' '$3=="185.213.154.201"'
```

How many TLS sessions to 194.165.16.56 have an empty server_name field?

82

The endpoint is `/upload/data`, and connection ID `CmQW1d3iuUHHdGqYe7` serves as the central anchor for this investigation. `ssl.log` presents the exact opposite scenario: while HTTP reveals complete payload details, TLS conceals nearly everything—making the absence of cleartext data the key indicator itself.

```
zeek-cut id.resp_h server_name < logs/ssl.log | sort | uniq -c
```

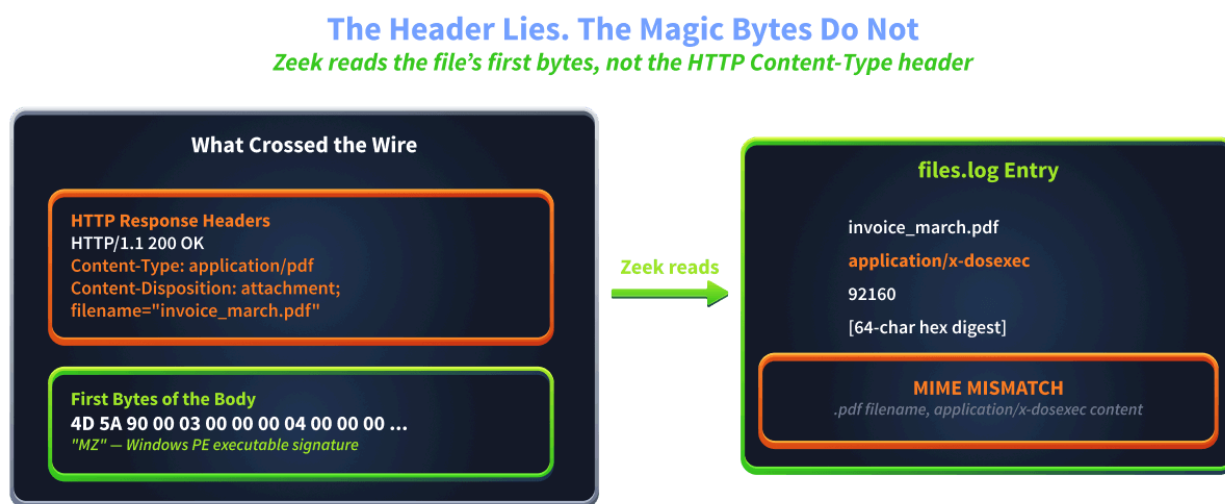
```
ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.resp_h method host uri user_agent request_body_len status_code < logs/http.log
1763088600.061000      CIEtDD4xl10B8efmf      10.14.22.88      185.220.101.45 GET      fileshare.corp-helpdesk.net      /downloads/invoice_march.pdf      \x2d 0 200
1763090107.081000      CmQW1d3iuUHHdGqYe7      10.14.22.88      185.213.154.201 POST      185.213.154.201 /upload/data      -      5348721 200
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h method uri user_agent request_body_len < logs/http.log | awk -F'\t' '$5=="-"'
10.14.22.88      185.213.154.201 POST      /upload/data      -      5348721
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h id.resp_h version server_name < logs/ssl.log | head -5
1763087880.041000      10.14.22.88      194.165.16.56      TLSv12      -
1763087942.429062      10.14.22.88      194.165.16.56      TLSv12      -
1763088002.959872      10.14.22.88      194.165.16.56      TLSv12      -
1763088062.385944      10.14.22.88      194.165.16.56      TLSv12      -
1763088118.000790      10.14.22.88      194.165.16.56      TLSv12      -
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h server_name < logs/ssl.log | awk -F'\t' '$2=="194.165.16.56"' | sort -u
10.14.22.88      194.165.16.56      -
ubuntu@tryhackme:~$ zeek-cut id.orig_h id.resp_h server_name < logs/ssl.log | awk -F'\t' '$2=="194.165.16.56" && $3=="-" {c++} END{print c+0}'
82
ubuntu@tryhackme:~$ grep -E "(^#)|(CmQW1d3iuUHHdGqYe7)" logs/files.log | zeek-cut source mime_type filename total_bytes
HTTP      application/zip backup_archive.zip      5348721
HTTP      -      -      7
```

Every one of the 82 connections to the C2 server lacks an SNI host value. Note that Zeek uses a dash (`-`) rather than a blank value for missing entries, which is crucial for crafting accurate filters. Standard browsers always transmit an SNI extension to support virtual hosting, whereas a C2 beacon targeting a raw IP address omits it entirely. Consequently, 82 bare TLS handshakes directed at a single IP provide a much stronger threat indicator than any individual certificate attribute.

Task 5 - File Metadata and Hash Correlation

What files.log Contains: Zeek's file analysis framework identifies files transferred over any supported protocol (HTTP, FTP, SMTP, SMB, SSL) and writes a row to `files.log` for each one. The fields we will care about most are `uid` (the connection that carried the file), `source` (the protocol that delivered it), `mime_type` (Zeek's identification of the file from its bytes), `filename` (the name from protocol metadata such as the HTTP `Content-Disposition` header), `total_bytes`, and `sha256` (the cryptographic hash of the file's contents).

The most important property of `files.log` for an analyst is that the `mime_type` is computed from the file's actual bytes, not from any header the server sent. The HTTP `Content-Type` header is a claim made by the server, while Zeek's MIME analyser reads the first bytes of the file directly and determines the type from the magic-byte signature. The header can lie, but the magic bytes do not, and that distinction is what makes the next exercise possible.



In a corporate Windows environment, this single row is enough to escalate.

The Exfil Archive: Let us start with the first look at `files.log`, as shown below:

```
ubuntu@tryhackme:~$ zeek-cut ts uid source mime_type filename
total_bytes sha256 < logs/files.log
1763088600.062000 CdxY4e40hSKGjp4mDf HTTP application/<
REDACTED> invoice_march.pdf 92160 -
1763090107.081000 CmQW1d3iuUHHDgqYe7 HTTP application/z
```

```
ip backup_<REDACTED>.zip 5348721 459b0165d7f2e5577d60cd8c1
244daded93f4457ed4a89983e5e36c490402bec
1763090109.469327 CmQW1d3iuUHHDgqYe7 HTTP - - 7 -
```

Looking at the output, three rows are visible. The first is a download we will examine in a moment, while the second and third share the same uid `CmQW1d3iuUHHDgqYe7`, which we recognise from `conn.log` as the 5.3 MB POST. The same connection produced two file rows because Zeek tracks the request body (the file the client uploaded) and the response body (the small acknowledgement the server returned) as separate file objects.

The row we will care about for the exfiltration is the second one. Looking at the fields, we see source `HTTP`, mime type `application/zip`, filename `backup_<REDACTED>.zip`, size 5,348,721 bytes, and a populated SHA256 value. The filename is what the attacker put in the HTTP `Content-Disposition` header during the upload, while the mime type is what Zeek computed from the first four bytes of the body (`50 4B 03 04`, the ZIP magic). The file type and byte count align, and the SHA256 gives us a stable fingerprint for the artefact that left the network in this connection. The filename remains redacted in the walkthrough output because learners derive the full value directly from the VM log.

When the Filename Lies: The first row in the previous output is more interesting than the filename suggests, so let us filter for files whose filename ends in a benign-looking extension: MIMEMismatch Hunt

```
ubuntu@tryhackme:~$ zeek-cut filename mime_type total_bytes <
logs/files.log | awk -F'\t' '$1~/\.(pdf|jpg|png|gif|doc)$/ '
invoice_march.pdf application/<REDACTED> 92160
```

Looking at the output, the filename ends in `.pdf` and is named to look like a finance document, but the mime type Zeek reports is `application/<REDACTED>`, which is the magic-byte signature for a Windows PE executable (`4D 5A`, the `MZ` header). The two fields disagree, and that disagreement is the entire signal. The HTTP server delivered this file with whatever `Content-Type` header it chose, and may have served it as `application/pdf` to look benign, but Zeek read the first bytes of the body, saw `MZ`, and reported the truth.

This file was downloaded by `WKST-FINANCE-04` from `185.220.101.45` shortly before the beacon traffic intensified. We have not identified what the executable does, but the MIME mismatch alone justifies escalating the host to incident response and pulling the file for sandbox analysis. From a Zeek perspective the analyst does not need to know the malware family in advance, because they only need to know the network claimed to deliver a PDF and actually delivered an executable, and `files.log` answers that with one row.

Hash Correlation Against the Threat Feed: `files.log` carries a SHA256 for every file it observes when hashing is enabled. Let us pull the SHA256 for the exfil archive: Capture the Exfil SHA256

```
ubuntu@tryhackme:~$ grep -E "(^#)|(CmQWld3iuUHHDgqYe7)" logs/
files.log | zeek-cut sha256 filename total_bytes | awk -F'\t'
'$2=="backup_archive.zip"'
459b0165d7f2e5577d60cd8c1244daded93f4457ed4a89983e5e36c490402
bec backup_archive.zip 5348721
```

A 64-character hex string sits in the SHA256 column, and that hash is the fingerprint of the exact bytes that left the network. Two analysts running the same query on the same dataset would compute the same value, which makes the hash a reliable handoff to threat intelligence and to detection engineering.

In a production investigation we would submit the hash to VirusTotal, MalwareBazaar, or an internal threat-feed platform and pull back any matches. The lab box has no outbound network by design (a learner cannot leak the dataset to a public service), so we will use a local reference feed at `threat-feed.csv` instead. Let us capture the hash and grep it against the CSV: Threat-Feed Lookup

```
ubuntu@tryhackme:~$ HASH=$(grep -E "(^#)|(CmQWld3iuUHHDgqYe7)" logs/files.log | zeek-cut sha256 filename total_bytes | awk -F'\t' '$2=="backup_archive.zip" {print $1; exit}')
ubuntu@tryhackme:~$ grep "$HASH" threat-feed.csv
459b0165d7f2e5577d60cd8c1244daded93f4457ed4a89983e5e36c490402
bec,Cobalt<REDACTED>,2025-11-10,high
```

A single matching row comes back, and the hash maps to family `Cobalt<REDACTED>` with a high-confidence rating and a first-seen date a few days before the incident. Combined with the empty SNI on the C2 beacons, the AS44477 destination, and the lateral SMB scan from the same host, the verdict is no longer ambiguous: the Finance subnet has a Cobalt Strike implant on `WKST-FINANCE-04`, the operator has staged data for upload, and a 5.3 MB ZIP carrying that data has already left the network.

What `files.log` added to the picture is that the file is a real ZIP archive, it has a known SHA256, and that SHA256 sits on a threat feed associated with a named malware family. The next move is to pivot all four log types together for a full session reconstruction, which is what the cross-log correlation step covers.

Answer the questions below

What is the value of the `filename` field in `files.log` for the file Zeek extracted from the 5.3 MB POST?

backup_archive.zip

The outbound exfiltration POST contains the file `backup_archive.zip` (`application/zip`), which is 5,348,721 bytes in size and originates from the local host (`is_orig=T`).

```
ubuntu@tryhackme:~$ grep -E "(^#)|(CmQW1d3iuUHHdGqYe7)" logs/files.log | zeek-cut source mime_type filename total_bytes
HTTP application/zip backup_archive.zip 5348721
HTTP - - 7
```

A file in `files.log` has `filename: invoice_march.pdf` but a different `mime_type`. What MIME type does Zeek report for the file content?

backup_archive.zip

```

ubuntu@tryhackme:~$ zeek-cut ts uid source mime_type filename total_bytes sha256
< logs/files.log
1763088600.062000      CIEtDD4xl10B8efmf      HTTP      application/x-dosexec i
nvoice_march.pdf      92160      7fbfa8adcb61123d10b8bc5b146308bbaf3a680dcc7ebf7d
8bfadd6b9356d805
1763090107.081000      CmQW1d3iuUHHDgqYe7     HTTP      application/zip backup_a
rchive.zip            5348721 459b0165d7f2e5577d60cd8c1244daded93f4457ed4a89983e5e36c4
90402bec
1763090109.469327      CmQW1d3iuUHHDgqYe7     HTTP      -          -          7          c
b0a78950e0d967dbce55cf8f4217d02d87c7f35fc0144af39fe7ffa5a3dc933
ubuntu@tryhackme:~$ zeek-cut filename mime_type total_bytes < logs/files.log | a
wk -F'\t' '$1~/\.(pdf|jpg|png|gif|doc)$/'
invoice_march.pdf      application/x-dosexec    92160

```

The incoming file `invoice_march.pdf` (92,160 bytes, `is_orig=F`) is actually a Windows executable despite its PDF extension. Zeek flagged it as `application/x-dosexec` by analyzing its magic bytes rather than its filename—marking the initial payload download that triggered the entire incident.

Take the SHA256 of `backup_archive.zip` from `files.log` and search for it in `/home/ubuntu/threat-feed.csv`. What family name is in the matching row?

CobaltStrikeStager

```

ubuntu@tryhackme:~$ grep -E "(^#)|(CmQW1d3iuUHHDgqYe7)" logs/files.log | zeek-cu
t sha256 filename total_bytes | awk -F'\t' '$2=="backup_archive.zip"'
459b0165d7f2e5577d60cd8c1244daded93f4457ed4a89983e5e36c490402bec      backup_a
rchive.zip            5348721
ubuntu@tryhackme:~$ HASH=$(grep -E "(^#)|(CmQW1d3iuUHHDgqYe7)" logs/files.log |
zeek-cut sha256 filename total_bytes | awk -F'\t' '$2=="backup_archive.zip" {pri
nt $1; exit}')
ubuntu@tryhackme:~$ grep "$HASH" threat-feed.csv
459b0165d7f2e5577d60cd8c1244daded93f4457ed4a89983e5e36c490402bec,CobaltStrikeSta
ger,2025-11-10,high

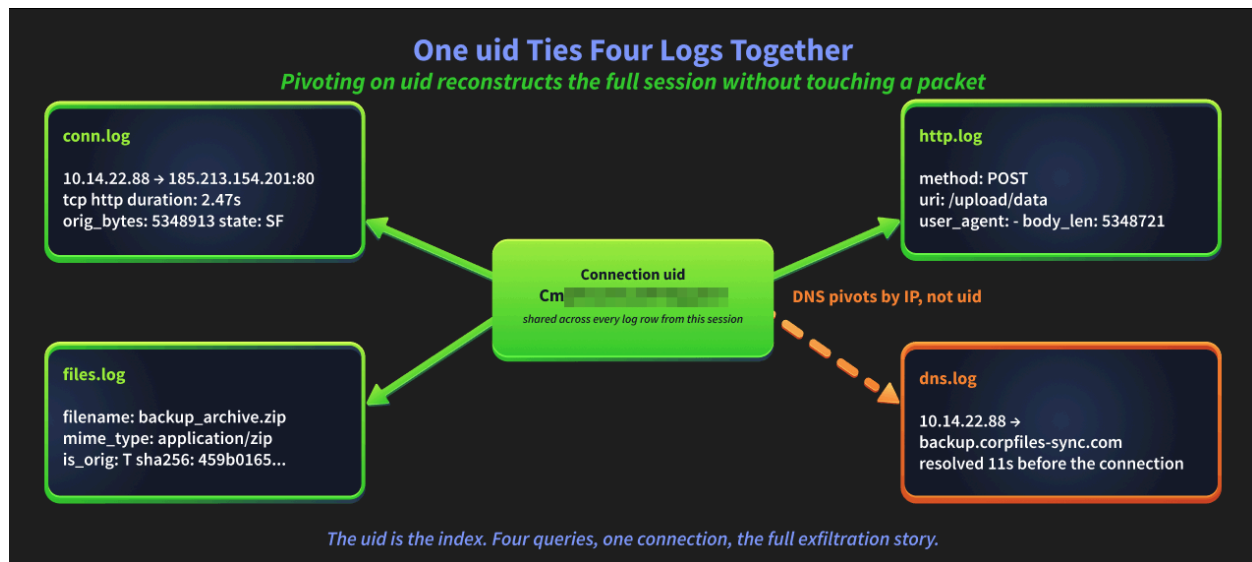
```

Task 6 - Cross-Log Correlation

The UID Is the Key: Each Zeek log we have looked at so far tells one part of the story. `conn.log` shows the connection, `http.log` shows the HTTP transaction inside it, `ssl.log` shows the TLS handshake, and `files.log` shows the file that crossed the wire. Rows that belong to the same network session share one column: `uid`. That value lets an analyst pivot between log types without opening the packet capture.

Cobalt Strike beacons, exfiltration POSTs, and lateral SMB scans all leave their traces in different logs. The job is to connect those traces carefully, using the

fields Zeek provides rather than guessing from a single log in isolation.



Starting in conn.log: Let us start where every Zeek investigation starts: `conn.log`. The connection we have flagged from the previous tasks is the large outbound HTTP POST to `185.213.154.201`, and we will pull the row in full, as shown below: Find the Suspicious Connection

```
ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.orig_p id.re
sp_h id.resp_p proto service duration orig_bytes resp_bytes c
onn_state < logs/conn.log | awk -F'\t' '$5=="185.213.154.20
1"'
1763090107.000000 <REDACTED_UID> 10.14.22.88 54891 185.2
13.154.201 80 tcp http 2.471327 5348913 110 SF
```

A single row comes back. Timestamp `1763090107` places the connection at 03:15:07 UTC. The source is `10.14.22.88`, the source port is `54891`, the destination is `185.213.154.201` on port `80`, the service is HTTP, the duration is 2.47 seconds, and `orig_bytes` shows 5,348,913 bytes outbound. The uid in the second column is the value we carry into the other logs.

Pivoting to http.log: Let us take the uid from `conn.log` and search `http.log` for the matching row: Pivot to http.log via uid

```
ubuntu@tryhackme:~$ grep -E "(^#)|(<UID_FROM_CONN_LOG>)" log
s/http.log | zeek-cut method host uri user_agent request_body
_len
POST      185.213.154.201 <REDACTED> -      5348721
```

The pivot returns one row. The method is `POST`, the Host header is the bare IP address, the User-Agent is unset, and the content is 5,348,721 bytes. This is the protocol-layer detail that `conn.log` could not give us. `conn.log` counted the bytes sent by the originator across the connection, while `http.log` gives us the HTTP request body length on its own.

That difference matters. The connection byte count is slightly larger because it includes HTTP request metadata as well as the body, while the HTTP body length is just the uploaded payload. Comparing the two values is a useful way to verify that the logs describe the same event at different levels of the stack.

Pivoting to files.log: The same uid produces multiple rows in `files.log` because the connection carried more than one file object: the client's upload and the small acknowledgment returned by the server. Pivot to files.log via uid

```
ubuntu@tryhackme:~$ grep -E "(^#)|(<UID_FROM_CONN_LOG>)" log
s/files.log | zeek-cut source is_orig mime_type filename tota
l_bytes
HTTP      <REDACTED> application/zip backup_archive.zip 53487
21
HTTP      <REDACTED> -      -      7
```

Two rows come back. The larger row contains the uploaded ZIP archive, and the smaller row contains the responder's acknowledgment body. The `is_orig` field indicates the direction: whether the originator or the responder sent the file body. In this case, the byte counts and filenames make the transfer shape clear even before we write the final incident note.

Backward Correlation From IP to Domain: The uid does not appear in `dns.log` for this case because the DNS query that resolved `185.213.154.201` happened in a separate UDP session with its own uid. The pivot for DNS is therefore based on IP

address rather than uid. Let us search `dns.log` for any query whose `answers` field contained the destination IP: DNS Backward Correlation

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h query answers < log
s/dns.log | grep "185.213.154.201"
1763090096.000000 10.14.22.88 backup.<REDACTED> 185.213.1
54.201
```

A single row comes back. Source `10.14.22.88` queried a backup-themed domain at timestamp `1763090096`, and the resolver returned `185.213.154.201`. Eleven seconds later, the HTTP POST to that IP began. The DNS resolution and the connection are temporally tied together, even though they do not share a uid.

Backward correlation from a destination IP to the domain that resolved it is one of the most valuable techniques in network analysis. It turns a faceless IP into a name we can map to attacker infrastructure, search across the rest of the dataset, and hand to threat intelligence as an indicator.

The Full Picture: Four logs now describe the same event from different angles. `conn.log` gives the destination, service, timing, and connection-level byte counts. `http.log` confirms the POST method, the missing User-Agent, and the request body length. `files.log` identifies the transferred file objects and their direction. `dns.log` ties the destination IP back to a domain resolved immediately before the POST.

The value of `uid` is not that it gives one answer by itself. Its value is that it lets us prove that separate rows describe the same network session. Once the rows are stitched together, the activity is no longer just a large connection or a suspicious HTTP request. It is a complete exfiltration event reconstructed from structured Zeek logs.

Answer the questions below

What is the difference, in bytes, between the `orig_bytes` value in `conn.log` and the `request_body_len` value in `http.log` for the exfiltration connection?

192

`conn.log` reports `orig_bytes` of 5,348,913 for that connection. `http.log` reports `request_body_len` of 5,348,721. The difference

is **192** bytes

```
ubuntu@tryhackme:~$ zeek-cut ts uid id.orig_h id.orig_p id.resp_h id.resp_p prot  
o service duration orig_bytes resp_bytes conn_state < logs/conn.log | awk -F'\t'  
'$5=="185.213.154.201"'
```

1763090107.000000	CmQW1d3iuUHHDgqYe7	10.14.22.88	54891	185.213.154.201	80	tcp	http	2.471327	5348913	110	SF
-------------------	--------------------	-------------	-------	-----------------	----	-----	------	----------	---------	-----	----

```
ubuntu@tryhackme:~$ grep -E "(^#)|(<UID_FROM_CONN_LOG>)" logs/http.log | zeek-cu  
t method host uri user_agent request_body_len  
ubuntu@tryhackme:~$ grep -E "(^#)|(<UID_FROM_CONN_LOG>)" logs/files.log | zeek-c  
ut source is_orig mime_type filename total_bytes  
ubuntu@tryhackme:~$ grep -E "(^#)|(CmQW1d3iuUHHDgqYe7)" logs/http.log | zeek-cut  
method host uri user_agent request_body_len  
POST 185.213.154.201 /upload/data - 5348721
```

How many bytes did the responder return as the file body in files.log for the same uid?

7

Pivoting the same **uid** into **files.log** gives the responder side: **7** bytes came back as the file body,

```
ubuntu@tryhackme:~$ grep -E "(^#)|(CmQW1d3iuUHHDgqYe7)" logs/files.log | zeek-cu  
t source is_orig mime_type filename total_bytes  
HTTP T application/zip backup_archive.zip 5348721  
HTTP F - - 7
```

Using the destination IP 185.213.154.201, search **dns.log** for the query that resolved to it. What is the queried domain?

backup.corpfiles-sync.com

the domain that resolved to 185.213.154.201 is backup.corpfiles-sync.com,

```
ubuntu@tryhackme:~$ zeek-cut ts id.orig_h query answers < logs/dns.log | gre  
p "185.213.154.201"
```

1763090096.000000	10.14.22.88	backup.corpfiles-sync.com	185.213.154.201
-------------------	-------------	---------------------------	-----------------

Task 7 - Writing a Detection Notice

Detection Scripting in Zeek: The previous tasks read logs Zeek had already produced. This one writes detection logic ourselves. Zeek's scripting language is

event-driven: every protocol analyser fires events as it parses traffic, and a script can subscribe to those events to add fields, filter rows, or generate alerts.

Detections raised by scripts go into `notice.log`, which is Zeek's equivalent of a Snort alert log but driven by full programming logic rather than rule syntax.

For Task 3 we identified two sources of long DNS queries on the wire: `WKST-FINANCE-04` running a tunnel to `exfil-channel.net`, and the internal DNS server `SRV-DNS-01` running DNSSEC validation against `dnssec-validation.local`. The first is malicious, while the second is legitimate. A naive detection rule fires on both, and our job in this task is to read a starter script that fires too broadly, run it to confirm the false-positive problem, then add a single condition that suppresses the legitimate source while keeping the genuine signal.

One Extra Check Removes the False Positives A two-line edit transforms a noisy detection into a usable one



Same data, same threshold, one extra check. That is detection tuning in two lines.

Reading the Starter Script: The starter script is at `scripts/dns_notice.zeek`. Let us read it from our home directory, as shown below: View the Starter Script

```
ubuntu@tryhackme:~$ cat scripts/dns_notice.zeek
```

```
@load base/frameworks/notice

module DNSTunnel;

export {
  redef enum Notice::Type += {
    DNS_Exfil_LongQuery
  }
}
```

```

    };
}

event dns_request(c: connection, msg: dns_msg, query: string, qtype: count, qclass: count)
{
    if ( |query| > 60 )
    {
        NOTICE([$note=DNS_Exfil_LongQuery,
                $conn=c,
                $msg=fmt("Long DNS query (%d chars): %s", |query|, query),
                $identifier=cat(c$id$orig_h),
                $suppress_for=5min]);
    }
}

```

Reading top to bottom, the `@load base/frameworks/notice` line pulls in the Notice framework, which provides the `NOTICE()` function we will call. The `module DNSTunnel;` line gives us a namespace so the notice type does not collide with anything else loaded on the same Zeek instance. The `redef enum Notice::Type +=` block registers a new notice type called `DNS_Exfil_LongQuery`, which is what `notice.log` records as the trigger.

The detection itself is a single event handler. The `event dns_request(...)` line subscribes to the DNS query event, which fires once for every DNS query Zeek observes. The `if ( |query| > 60 )` line tests the length of the query string. The `NOTICE(...)` call writes a row to `notice.log` with the connection record (`$conn=c`), a formatted message (`$msg`), a per-source-IP identifier for deduplication (`$identifier`), and a five-minute suppression window (`$suppress_for`). There is no condition here that excludes the internal DNS server. Any DNS query longer than 60 characters from any source produces a notice.

Running the Starter: Let us process the PCAP through Zeek with the starter script loaded. We will use a separate working directory under `/tmp` so the new logs do not overwrite the staged ones, as shown below: Run the Starter Script

```

ubuntu@tryhackme:~$ mkdir -p /tmp/zeek-task7 && cd /tmp/zeek-task7
ubuntu@tryhackme:/tmp/zeek-task7$ zeek -r ~/pcaps/finance-sensor.pcap ~/scripts/dns_notice.zeek

```

```
ubuntu@tryhackme:/tmp/zeek-task7$ ls -la notice.log
-rw-rw-r-- 1 ubuntu ubuntu 7189 May  4 18:21 notice.log
```

A `notice.log` exists in our working directory. Let us read a sample of the rows by source IP: Read Notices by Source

```
ubuntu@tryhackme:/tmp/zeek-task7$ zeek-cut id.orig_h note msg
< notice.log | sort -u | head -5
10.0.0.53   DNSTunnel::DNS_Exfil_LongQuery  Long DNS query (8
0 chars): 149f85cfb17f970bbc60bd02dd5ad4cbf781f6defd63bf4ffb7
85e3a.dnssec-validation.local
10.0.0.53   DNSTunnel::DNS_Exfil_LongQuery  Long DNS query (8
0 chars): 2fee0a735124788c213c42a6633abef777066836f39486979e3
1aa60.dnssec-validation.local
10.0.0.53   DNSTunnel::DNS_Exfil_LongQuery  Long DNS query (8
0 chars): 4d74152717deebcad8cbc36e6ab07a88a96eb93bd9ef85ad001
86e4a.dnssec-validation.local
10.0.0.53   DNSTunnel::DNS_Exfil_LongQuery  Long DNS query (8
0 chars): 5c0ed2a226ab3c6f3eb920b22982dd1b4b9e4947f45bfed73a1
cf5ea.dnssec-validation.local
10.0.0.53   DNSTunnel::DNS_Exfil_LongQuery  Long DNS query (8
0 chars): 5d7c30d21ba531df669350745eb6e7e50e1672fe312e592b146
2ac2b.dnssec-validation.local
```

The sample shows the internal DNS server producing long DNSSEC validation queries under `dnssec-validation.local`. These are not malicious; they are the false positives we identified in Task 3. The starter rule is too broad because it only checks query length. Now count the total number of notices created by the starter script: Count Starter Notices

```
ubuntu@tryhackme:/tmp/zeek-task7$ zeek-cut id.orig_h < notic
e.log | wc -l
<COUNT>
```

The total includes both the legitimate DNSSEC activity and the genuine tunnel activity. We need to tune the script so that the legitimate source is suppressed

while the suspicious workstation remains visible.

Applying the Suppression: Let us add a single condition at the top of the event handler. If the querying host is the internal DNS server, the script will return early and skip the `NOTICE()` call. The modified event handler looks like this:

```
event dns_request(c: connection, msg: dns_msg, query: string, qtype: count, qclass: count)
{
    if ( c$id$orig_h == 10.0.0.53 )
        return;

    if ( |query| > 60 )
    {
        NOTICE([$note=DNS_Exfil_LongQuery,
                $conn=c,
                $msg=fmt("Long DNS query (%d chars): %s", |query|, query),
                $identifier=cat(c$id$orig_h),
                $suppress_for=5min]);
    }
}
```

The new two-line check tests the connection's originator IP against the literal `10.0.0.53` and returns from the event handler before the length check runs. Any query from that source skips notice generation entirely, while every other source runs the original logic. Save the modified version as `dns_notice_modified.zEEK` in the working directory and re-process the PCAP: Re-run with the Modified Script

```
ubuntu@tryhackme:/tmp/zeek-task7$ rm -f *.log
ubuntu@tryhackme:/tmp/zeek-task7$ zeek -r ~/pcaps/finance-sensor.pcap dns_notice_modified.zEEK
ubuntu@tryhackme:/tmp/zeek-task7$ ls -la notice.log
-rw-rw-r-- 1 ubuntu ubuntu 3446 May  4 18:21 notice.log
```

The `notice.log` regenerated and is roughly half the size it was before. The DNSSEC false positives are gone, and the remaining rows are the genuine signal from the workstation.

Confirming the Fix: Let us count the notices after the suppression has been applied: Count Tuned Notices

```
ubuntu@tryhackme:/tmp/zeek-task7$ zeek-cut id.orig_h < notice.log | wc -l  
<COUNT>
```

The tuned count should be lower than the starter count because the legitimate DNSSEC source has been removed. The remaining notices are the suspicious long-query activity from the workstation. We can also list the remaining unique sources to confirm that the tuned rule is focused on the genuine signal: Confirm Remaining Source

```
ubuntu@tryhackme:/tmp/zeek-task7$ zeek-cut id.orig_h < notice.log | sort -u  
<REMAINING_SOURCE>
```

The detection now reports the activity that matters and stays quiet on the activity that does not. That is what a tuned rule looks like in practice: same data, same threshold, one additional condition, and a much higher signal-to-noise ratio.

How the Suppression Attribute Works: One more piece of the script is worth pulling out before we close. Looking back at the original `NOTICE()` call, it ends with `$identifier=cat(c$id$orig_h)`, paired with `$suppress_for=5min`. The `$identifier` is the deduplication key, which in this script is just the source IP.

The `$suppress_for` value controls how long Zeek holds back another notice with the same identifier before letting it fire again.

With those two attributes set, Zeek emits at most one notice per source IP every five minutes, even if the host fires many long queries in that window. The underlying activity is still visible in `dns.log`, but `notice.log` stays readable enough for an analyst to investigate. The Notice framework, the event-driven script, and the suppression control together turn a noisy detection into a usable one.

Answer the questions below

Before tuning the script, how many total notices does `dns_notice.zeek` generate?

23

```

ubuntu@tryhackme:~$ mkdir -p /tmp/zeek-task7 && cd /tmp/zeek-task7
ubuntu@tryhackme:/tmp/zeek-task7$ zeek -r ~/pcaps/finance-sensor.pcap ~/scripts/
dns_notice.zeek
ubuntu@tryhackme:/tmp/zeek-task7$ -la notice.log
Command '-la' not found, did you mean:
  command 'tla' from deb tla (1.3.5+dfsg1-2.1)
Try: sudo apt install <deb name>
ubuntu@tryhackme:/tmp/zeek-task7$ ls -la notice.log
-rw-rw-r-- 1 ubuntu ubuntu 7187 Sep  2 07:37 notice.log
ubuntu@tryhackme:/tmp/zeek-task7$ zeek-cut id.orig_h note msg < notice.log | sor
t -u | head -5
10.0.0.53      DSTunnel::DNS_Exfil_LongQuery  Long DNS query (80 chars): 149f8
5cfb17f970bbc60bd02dd5ad4cbf781f6defd63bf4ffb785e3a.dnssec-validation.local
10.0.0.53      DSTunnel::DNS_Exfil_LongQuery  Long DNS query (80 chars): 2fee0
a735124788c213c42a6633abef777066836f39486979e31aa60.dnssec-validation.local
10.0.0.53      DSTunnel::DNS_Exfil_LongQuery  Long DNS query (80 chars): 4d741
52717deebcad8cbc36e6ab07a88a96eb93bd9ef85ad00186e4a.dnssec-validation.local
10.0.0.53      DSTunnel::DNS_Exfil_LongQuery  Long DNS query (80 chars): 5c0ed
2a226ab3c6f3eb920b22982dd1b4b9e4947f45bfed73a1cf5ea.dnssec-validation.local
10.0.0.53      DSTunnel::DNS_Exfil_LongQuery  Long DNS query (80 chars): 5d7c3
0d21ba531df669350745eb6e7e50e1672fe312e592b1462ac2b.dnssec-validation.local
ubuntu@tryhackme:/tmp/zeek-task7$ zeek-cut id.orig_h < notice.log | wc -l
23

```

After suppressing 10.0.0.53 and rerunning Zeek, how many notices remain?

15

Task 8 - Conclusion

We have now walked through a complete Zeek-led investigation, starting with high-level connection metadata and ending with a tuned notice script. The important lesson is not that one log file solved the case. The case became clear because each log confirmed a different part of the same activity.

- `conn.log` gave us the shape of the traffic: repeated outbound sessions, unusual connection states, destination reuse, and byte counts that pointed towards beaconing and upload behaviour. That gave us the first investigative thread.
- `dns.log` added naming context. It showed which domains mapped to suspicious destinations, where long TXT queries appeared, and which DNS events were actually legitimate validation traffic rather than exfiltration. That distinction matters because analysts need to reduce noise without hiding the signal.

- `http.log` showed the application-layer behaviour behind the transfer. We used the HTTP method, host, URI, request body length, and user agent fields to identify the upload path and connect it back to the suspicious connection record.
- `ssl.log` gave us the encrypted-session evidence. Even when payloads were not visible, the TLS metadata still exposed useful behaviour: repeated sessions to the same destination, empty SNI values, and timing patterns that matched the rest of the investigation.
- `files.log` confirmed that the uploaded object was not what its filename suggested. By checking the filename, MIME type, byte counts, and SHA256 hash, we could prove that the archive was suspicious and then match it against the local threat intelligence feed.

The key field across the investigation was `uid`. It let us move from one log to another without guessing. Once we had the right UID, we could follow the same network event through connection metadata, HTTP details, TLS details, and extracted file evidence.

What You Should Take Away

- Start broad with `conn.log`, then pivot into protocol-specific logs when the traffic shape looks suspicious.
- Use `dns.log` to explain where destinations came from, but verify whether the DNS behaviour is malicious or expected infrastructure noise.
- Use `http.log` and `ssl.log` to prove what the client attempted to do, even when the payload itself is encrypted.
- Use `files.log` when the investigation involves uploads, downloads, MIME mismatches, hashes, or file names that do not match the content.
- Use Zeek scripting to turn repeated investigative logic into repeatable detection, then tune the script so known-good activity does not drown out the real alert.

In a SOC, this workflow is useful because it keeps the investigation evidence-led. We are not relying on one suspicious field or one screenshot. We are building a chain of observations that support each other: connection behaviour, name resolution, protocol metadata, file evidence, and tuned detection logic.